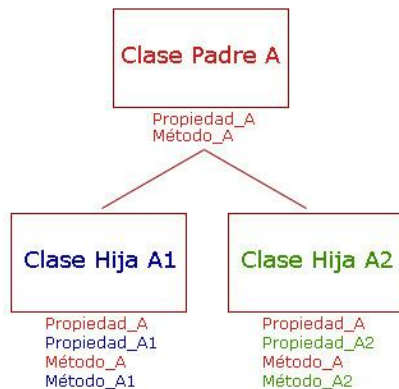


Herencia en Python

1. ¿Qué es la herencia?

La herencia permite crear nuevas clases a partir de otras existentes. La clase original se conoce como clase padre y la nueva como clase hija. Gracias a este mecanismo es posible reutilizar código, mantener programas organizados y facilitar su crecimiento.

Herencia de Clases

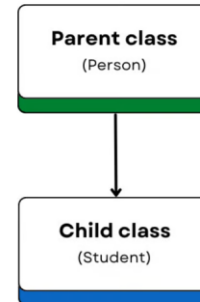


2. Tipos de herencia

Python soporta herencia única y múltiple. Conceptualmente también existen la herencia multinivel, jerárquica e híbrida. Cada una representa una forma distinta en que las clases pueden relacionarse.

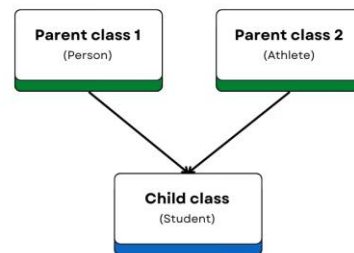
Herencia única

SINGLE INHERITANCE



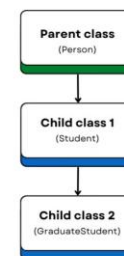
Herencia múltiple

MULTIPLE INHERITANCE



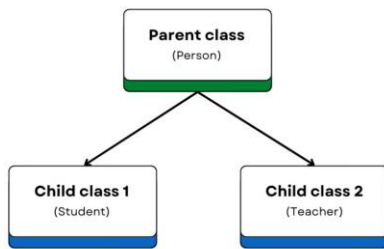
Herencia multinivel

MULTILEVEL INHERITANCE



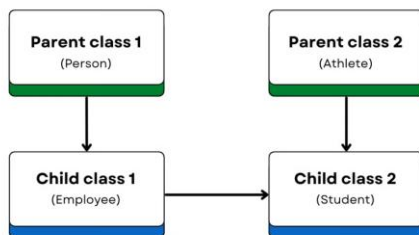
Herencia jerárquica

HIERARCHICAL INHERITANCE



Herencia híbrida

HYBRID INHERITANCE



3. Crear una clase padre y una clase hija

Primero se define la clase padre con atributos y métodos comunes. Luego la clase hija hereda de ella y puede agregar nuevos atributos o comportamientos.

```
# Definición de la clase padre
class Persona:
    def __init__(self, nombre, identificacion):
        self.nombre = nombre
        self.identificacion = identificacion

    def mostrar_informacion(self):
        return f"Nombre: {self.nombre}, ID: {self.identificacion}"

# Definición de la clase hija
class Estudiante(Persona):
    def estudiar(self):
        return f"{self.nombre} está estudiando."
```

4. Ventajas y limitaciones

Ventajas

- **Reutilización:** Con la herencia puedes escribir código una vez en la clase padre y reutilizarlo en las clases hijas.
- **Simplicidad:** La herencia modela las relaciones con claridad.
- **Escalabilidad:** También añade nuevas funciones o clases hijo sin afectar al código existente.

Limitaciones

- **Complejidad:** Demasiados niveles de herencia pueden hacer que el código sea difícil de seguir.
- **Dependencia:** Los cambios en una clase padre pueden afectar involuntariamente a todas las subclases.
- **Uso indebido:** Utilizar la herencia cuando no es lo más adecuado puede complicar los diseños.

5. Sobreescritura de métodos

Una clase hija puede redefinir un método heredado para adaptarlo a sus necesidades. Esto permite que objetos diferentes respondan de forma distinta al mismo método.

```
# Definición de la clase padre
class Persona:
    def __init__(self, nombre, id):
        self.nombre = nombre
        self.id = id

    def obtener_detalle(self):
        return f"Nombre: {self.nombre}, ID: {self.id}"
```

```

# Definición de la clase hija
class Estudiante(Persona):
    def __init__(self, nombre, id, calificacion):
        # Llama al constructor de la clase padre
        super().__init__(nombre, id)
        self.calificacion = calificacion

# Sobrescribe el método de la clase padre
def obtener_detalle(self):
    return (
        f"Nombre: {self.nombre}, "
        f"Id: {self.identificacion}, "
        f"Calificación: {self.calificacion}"
    )

```

6. Uso de super()

La función `super()` llama al constructor o métodos de la clase padre, evitando duplicar código y garantizando una correcta inicialización.

```

# Definición de la clase padre
class Persona:
    def __init__(self, nombre, id):
        self.nombre = nombre
        self.id = id

# Definición de la clase hija
class Estudiante(Persona):
    def __init__(self, nombre, id, calificacion):
        # Utiliza super() para llamar al
        # constructor de la clase padre
        super().__init__(nombre, id)
        # Inicializa el atributo propio de la
        # clase
        self.calificacion = calificacion

```

7. Polimorfismo

Permite utilizar una misma función con objetos de diferentes clases. Si todos implementan el mismo método, Python ejecutará automáticamente la versión correspondiente a cada objeto.

```

# Definición de la clase padre
class Persona:
    def obtener_detalle(self):
        return "Detalles de una persona."

# Definición de la clase Estudiante
class Estudiante(Persona):
    def obtener_detalle(self):
        return "Detalles de un estudiante."

# Definición de la clase Profesor
class Profesor(Persona):
    def obtener_detalle(self):
        return "Detalles de un profesor."

```

Actividad 2: Sistema de Personas

Una universidad desea desarrollar un sistema sencillo para administrar la información de las personas que forman parte de la institución. Utilice los conceptos de **herencia** para implementar la siguiente solución.

Requisitos:

- Cree una clase `Persona` con los atributos:
 - nombre
 - identificación
- Agregue el método `obtener_detalle()` que devuelva la información de la persona.
- Cree una clase `Estudiante` que herede de `Persona` e incluya el atributo `carrera`.
- Cree una clase `Profesor` que herede de `Persona` e incluya el atributo `departamento`.
- Sobrescriba el método `obtener_detalle()` en las clases `Estudiante` y `Profesor`.
- Implemente una función llamada `imprimir_detalle(persona)` que reciba un objeto e imprima el resultado de `obtener_detalle()`.
- En el programa principal:
 - Cree 2 estudiantes y 2 profesores.
 - Guarde todos los objetos en una lista.
 - Recorra la lista utilizando un ciclo `for` e imprima la información de cada objeto.